Fine-tuning, Evaluation, and Ensemble of Cross-Encoder Models for Document Reranking

Ataklti Kidanemariam (s4590821) and Benard Adala Wanyande (s4581733)

Leiden University, Leiden, Netherlands

Abstract. This report details the process of fine-tuning and evaluating three different cross-encoder models for document reranking, and investigates ensemble methods and query expansion. Three models: `cross-encoder/ms-marco-MiniLM-L-2-v2`, `cross-encoder/ms-marco-TinyBERT-L-2-v2`, and `distilroberta-base`, were fine-tuned on the MS MARCO passage dataset for approximately one hour each. Evaluation on TREC DL 2019 showed varying performance, with TinyBERT and MiniLM outperforming DistilRoberta. Five ensemble methods (CombSUM, CombMNZ, RRF, BordaFuse, LogISRFuse) were applied to combine model outputs. RRF and BordaFuse enhanced performance over the best single model, demonstrating fusion utility. Further analysis with RRF on pairwise model combinations explored how different model pairings affected results. Finally, the impact of LLM-based query expansion on reranking was tested on a subset of queries, revealing a significant decrease in performance across all models, highlighting challenges in this integration.

1 Introduction

Information Retrieval (IR) systems often use a multi-stage ranking process: an initial efficient retrieval followed by a more accurate reranking. Cross-encoders are powerful for reranking by modeling query-document interactions.

This study focuses on fine-tuning and evaluating three Transformer-based cross-encoder models for passage reranking using the MS MARCO dataset [1]. Models were fine-tuned for a standardized duration (1 hour) for comparison. Evaluation was conducted on the TREC Deep Learning (DL) 2019 dataset [2], using NDCG@10, Recall@100, and MAP@1000.

Subsequently, we investigate ensemble methods (fusion) using the `ranx` library [5] to combine the outputs of the three models, aiming for further performance improvements. This includes fusing all three models and all pairwise combinations.

Finally, we examine the impact of Query Expansion (QE) using Large Language Models (LLMs) on reranking performance, evaluating the cross-encoders on a subset of TREC DL 2019 queries before and after LLM-based expansion.

2 Methodology

Three cross-encoder models were fine-tuned: `cross-encoder/ms-marco-MiniLM-L-2-v2`, `cross-encoder/ms-marco-TinyBERT-L-2-v2`, and `distilroberta-base`. Fine-tuning was performed using the `sentence-transformers` library [3] on the MS MARCO passage ranking dataset (triplets from `msmarco-qidpidtriples.rnd-shuf.train.tsv.gz`). The task was binary classification (relevant/irrelevant), predicting a relevance score.

Hyperparameters included: Batch Size 32, 1 Epoch (interrupted after 1 hour), Positive-to-Negative Ratio 4, Max Training Samples 5M, Warmup Steps 5k, Evaluation Steps 1k (on a development set), AdamW optimizer, default learning rate, and AMP enabled. Training duration varied by model size: MiniLM (58k steps), DistilRoberta (11k steps), TinyBERT (112k steps).

Evaluation was on the TREC DL 2019 test set, using the provided top-1000 candidate passages per query (`msmarco-passagetest2019-top1000.tsv.gz`) and relevance judgments (`2019qrels-pass.txt`). Metrics were NDCG@10, Recall@100, and MAP@1000, computed using `pytrec_eval` [4] via `ranx.evaluate`.

For ensembles, scores from individual runs were fused using `ranx.fuse` after min-max normalization. Five methods were tested: CombSUM, CombMNZ, RRF (k=60), BordaFuse, and LogISRFuse, first with all three models, then RRF (k=60) with all pairwise combinations.

For query expansion, a subset of 43 TREC DL 2019 queries was selected. LLM-expanded versions (details of LLM/prompt not specified here) were used for a second evaluation run, comparing performance against original queries on the same 43 queries' candidate passages and judgments. Cross-encoder input was truncated to 512 tokens.

3 Results

3.1 Individual Model Performance

Table 1. Individual model evaluation results on TREC DL 2019 (approx. 1hr fine-tuning).

Model	NDCG@10	Recall@100	MAP@1000	Training Steps
MiniLM-L-2-v2	0.6899	0.5068	0.4488	58,000
DistilRoberta-base	0.6310	0.4795	0.4074	11,000
TinyBERT-L-2-v2	0.6950	0.5049	0.4568	112,000

3.2 Performance of Ensemble Methods

Table 2. Evaluation results of ensemble methods fusing all three models on TREC DL 2019.

Ensemble Method (All 3 Models)	NDCG@10	Recall@100	MAP@1000
CombSUM	0.7051	0.5111	0.4585
CombMNZ	0.7051	0.5111	0.4585
RRF (k=60)	0.7109	0.5133	0.4658
BordaFuse	0.7098	0.5170	0.4653
LogISRFuse	0.6827	0.5086	0.4496

Table 3. Evaluation results of RRF (k=60) on pairwise model combinations on TREC DL 2019.

Models Fused with RRF (k=60)	NDCG@10	Recall@100	MAP@1000
MiniLM + DistilRoberta	0.6994	0.5092	0.4525
MiniLM + TinyBERT	0.7053	0.5112	0.4647
DistilRoberta + TinyBERT	0.6908	0.5090	0.4581

3.3 Query Expansion Results

4 Discussion

4.1 Individual Model Performance (Task 1)

TinyBERT and MiniLM, pre-trained on MS MARCO, significantly outperformed the general-purpose DistilRoberta model (Table 1). TinyBERT's higher training throughput within the fixed hour (112k steps vs 58k for MiniLM and 11k for DistilRoberta) likely contributed to its slightly better NDCG@10 and MAP@1000, while MiniLM achieved higher Recall@100. Domain-specific pre-training on a large dataset like MS MARCO is crucial for strong performance on related reranking tasks.

Table 4. Cross-encoder performance on 43 TREC DL 2019 queries, original vs. LLM-expanded.

Model	Query Type	NDCG@10	Recall@100	MAP@1000
MiniLM-L-2-v2	Original Query	55.52	37.28	32.66
	Expanded Query	33.93	27.04	18.03
DistilRoberta-base	Original Query	53.26	37.46	32.77
	Expanded Query	23.26	26.19	14.40
TinyBERT-L-2-v2	Original Query	61.84	39.50	35.88
	Expanded Query	28.05	28.30	18.52

4.2 Ensemble Methods Effectiveness (Task 2)

Fusing all three models using ensemble methods generally improved performance over the best single model (Table 2). Rank-based methods, RRF (0.7109 NDCG@10) and BordaFuse (0.7098 NDCG@10), were the most effective, surpassing TinyBERT's 0.6950. CombSUM and CombMNZ also improved performance but were slightly below RRF/BordaFuse. LogISRFuse performed poorly, even below the best individual models. This suggests that aggregating ranks was more effective than aggregating normalized scores for these specific model outputs, likely due to better handling of differing score distributions and leveraging complementary ranking information.

4.3 Pairwise Ensemble Analysis (Task 3)

Applying RRF (k=60) to pairwise combinations (Table 3) showed that fusing the two strongest models, MiniLM and TinyBERT, yielded the best pairwise result (0.7053 NDCG@10), outperforming each individually. Combinations involving DistilRoberta also generally improved over DistilRoberta alone, showing even a weaker model can contribute. However, RRF with all three models (0.7109 NDCG@10) slightly outperformed the best pairwise combination. This suggests that despite its lower overall effectiveness, DistilRoberta provided unique high-ranked documents for some queries that RRF could leverage when combined with the other two, highlighting the value of diversity for robust rank fusion.

5 Query Expansion by Prompting LLMs (Task 4)

Evaluating the cross-encoders on a subset of 43 queries revealed a significant drop in performance across all metrics when using LLM-expanded queries compared to original queries (Table 4). TinyBERT's NDCG@10 fell from 61.84 to 28.05, MiniLM's from 55.52 to 33.93, and DistilRoberta's from 53.26 to 23.26. This unexpected negative result indicates issues with this specific QE approach for cross-encoder reranking. Potential reasons include: 1) Suboptimal quality of LLM-generated expansions introducing noise; 2) Mismatch between expanded query characteristics and the models' training on original queries; 3) Increased query length leading to detrimental truncation within the cross-encoder's fixed context window; 4) Sophistication of cross-encoders may not benefit from simple term addition, and noisy expansions could degrade their semantic matching. This highlights that integrating LLM outputs into neural IR pipelines requires careful consideration beyond simple concatenation.

6 Conclusion

This study fine-tuned and evaluated three cross-encoder models for passage reranking. TinyBERT and MiniLM, pre-trained on MS MARCO, performed best individually. Ensemble methods, particularly RRF and BordaFuse, significantly improved performance by fusing the outputs of the fine-tuned models, demonstrating the benefit of combining different rankers. Analysis of pairwise fusion highlighted the strength of combining the top two models and the unexpected contribution of even a weaker model when using robust rank fusion. Finally, a specific LLM-based query expansion approach was found to severely degrade reranking performance across all models, underscoring the challenges of naively applying LLM outputs for QE in

sophisticated neural reranking systems. Future work should explore more refined LLM integration strategies and optimized fusion methods.

References

1. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., Deng, L.: MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. In: Workshop on Cognitive Computation: Integrating neural and symbolic approaches (NIPS 2016) (2016)
2. Craswell, N., Campos, D., Mitra, B., Yilmaz, E., Voorhees, E.: Overview of the TREC 2019 Deep Learning Track. In: Proceedings of the Twenty-Eighth Text REtrieval Conference (TREC 2019) (2019)
3. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP) (2019)
4. Van Gysel, C., de Rijke, M., Kanoulas, E.: pytrec_eval: An Official Python Interface to trec_eval. In: Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '18). Association for Computing Machinery, New York, NY, USA, 1277–1280 (2018)
5. Persia, F., Ciot, G., Cota, G., Silvestri, F., Pirozzi, F., Amen, T.: ranx: A Python Library for Evaluating and Comparing Ranking Models. In: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22). Association for Computing Machinery, New York, NY, USA, 2949–2953 (2022). https://doi.org/10.1145/3477495.3531745